

Recap Multi-Agent Systems

MARBEL agents & environments

Notice typical structure of agent questions

- General, basic questions that require you to *know* and *understand* basic concepts, related to:
 - Environment characteristics → See slides and discussion in all MARBEL Prog Guide Chapters
 - Types of actions, → See, e.g., Ch4.4
- Programming part, which requires you to *know how to program*:
 - Channels for processing percepts of different types → See, e.g., Ch5.2.1
 - Define basic knowledge (=Prolog) rules
 - Know how to incorporate pre-conditions of environment actions → See all Chapters & Ch4.4.
 - Complete conditions of action rules → See practice exam.

Sources you should look at: Lecture slides and Chapters in MARBEL Programming Guide on Canvas.

Agents and MARBEL: Basics

Properties of environments

- **Fully/partially observable:** Agents can (not) always see “everything” in the environment that is relevant for their decision-making. **Consequence:** If the environment is not completely observable the agent will need to explore / keep track of what agent sees.
- **Deterministic/stochastic:** The effects of an action in the environment are (not) completely determined by that action. **Consequence:** The effects of an action are (not) known / can (not) be predicted 100% (if not, agent needs to “see” what has changed).
- **Static/dynamic:** The environment (does not) only change when an agent performs an action. **Consequence:** if *dynamic*, agent needs to “see” what has changed *by the environment because things change that the agent did not change itself*.
- **Discrete/continuous:** If there is a limited number of percepts and actions the environment is discrete, otherwise it is continuous. **Consequence:** If continuous, agent needs to continuously monitor (“see”) how things change in the environment.
- **Single/multi-agents (mas):** There is one (or more = mas) agent(s) that are performing actions changing the environment. **Consequence:** In a mas, agents need to coordinate their actions.

Percept processing by channel handlers

Types of percepts and how to process them, see Ch5.2.1:

- Send once → use 'add' channel (only adds never removes info)
- Send always → use 'replace' channel (removes all previous information for specific percept and adds new info each agent cycle)
- Send on change → use 'update' channel (only removes old information for a percept when new information is received)

Percept processing

- Percept handlers (add, replace, update) are part of the agent definition in a mas2g file.
- If there is a need for (pre)processing of percept information, two options of where to do this:
 - In module “for init” if percept is “send once” (no use to do that repeatedly)
 - In module “for updates” if preprocessing may need to be done every agent cycle.
- Adding **initial** information like agendas, or other things that only need to be added once: do that in module “for init”.

Programming in MARBEL

Action rules

Order of action rules in module “for decisions” makes a difference:

- Default in “for decisions” module: rules are evaluated from top to bottom;
- First applicable rule (rule condition holds) is applied;
- As a rule of thumb(!), order rules for actions that are more important and should be ‘prioritized’ higher, before others; put rules for actions that are not important last. (Note that this is a ***RULE OF THUMB*** that works most of the time!).
- A rule with ‘true’ condition is always applicable and it does not make sense to put this before any other rule!

Evaluation order in modules for initializing (“for init”) and updating (“for updates”) different: all rules are evaluated AND applied if applicable.

See also Ch5.4.1

Make sure to get syntax right!

For example, remember to use **capitals at start of variable names!**

In condition part of rule, use “,” (the Prolog conjunction) to combine condition parts; for combining actions use “+”, for example:

```
if atBlock(BlockID), block(BlockID, ColorID, PlaceID), not(holding (_))  
    then pickUp(BlockID) + delete(block(BlockID, ColorID, PlaceID)).
```

Agent's (mental or cognitive) state or database

- MARBEL agent uses Prolog database (initialized by a Prolog file).
- Queries (conditions) of action rules are evaluated on agent's (current) database.
- Database is updated while agent is executed (by means of percepts and built-in actions).
- Using **insert(<PROLOG FACT>)** and **delete(<PROLOG FACT>)** to add and remove **facts** from database (Prolog **rules** cannot be added/removed!)

Reasoning about the order of rules

```
module towerBuilder {  
    if holding(X) then {  
        if constructiveMove(X, Y) then putdown(X, Y).  
        if true then putdown(X, table).  
    }  
  
    if constructiveMove(X, _) then pickup(X).  
  
    if misplaced(X) then pickup(X).  
  
    % AS A RULE OF THUMB, PUT RULES WHICH ONLY CHECK FOR true AT THE END  
    % ANY RULES AFTER RULE BELOW WOULD NEVER BE CHECKED...  
    if true then nil.  
}
```

Reasoning about the order of rules

```
module towerBuilder {  
    if holding(X) then {  
        if constructiveMove(X, Y) then putdown(X, Y).  
        if true then putdown(X, table).  
    }
```

% WHY DONT WE HAVE TO CHECK FOR not(holding(_)) HERE (it is a precondition of pickup action!)

```
    if constructiveMove(X, _) then pickup(X).
```

```
    ...
```

```
}
```

Reasoning about the order of rules

```
module towerBuilder {  
    if holding(X) then {  
        if constructiveMove(X, Y) then putdown(X, Y).  
        if true then putdown(X, table).  
    }  
}
```

% No need to check for not(holding(_)) because we never get here when holding(_) is true:

% (1) Nested rule above checks for holding(X), so this block of rules is entered when true;

% (2) The last rule of the two nested rules will always be applicable because condition is 'true'.

```
if constructiveMove(X, _) then pickup(X).
```

```
....
```

```
}
```

(How to) Prepare Yourself

Practice exam

- Do the Practice exam!